GitHub Desktop 브랜치 사용법

그래픽 파트 팀원을 위한 따라 하기 안내서

CK Semester Project

작성일 2026-11-02

목차

한눈에 보기 4
1. 브랜치와 저장의 차이 4
화면에서 먼저 볼 여섯 곳 5
2. 처음 한 번: 설치·로그인·Clone 6
설치하고 로그인하기 6
팀 저장소 가져오기 7
3. 매일 시작: 새 변경부터 받기 8
4. 작업할 브랜치 고르기 10
주요 브랜치 10
기존 브랜치로 이동하기 10
5. 새 브랜치 만들기 11
어느 브랜치에서 시작하나요? 11
생성하고 이름 확인하기 12
새 브랜치 처음 올리기 13
6. Unity 에서 작업하고 저장하기 13
7. 커밋할 파일 고르기 14
8. Summary 작성하고 Commit 하기 15
팀 커밋 제목 규칙 15
내 컴퓨터에 기록하기 15
9. Push 까지 마치고 확인하기 16
10. 브랜치를 옮길 때 변경이 남아 있다면 18
사라진 것처럼 보이는 작업 꺼내기 18
11. 꼭 지킬 주의사항 19
12. 자주 겪는 문제 20
브랜치를 옮겼더니 작업이 사라졌어요 20
Changes 에 아무것도 안 떠요 20
Push 가 안 돼요 20
충돌이 났어요 20

대용량 파일 오류가 나요 21
실수로 잘못된 브랜치에 작업했어요 21
Settings.json 만 계속 남아요 21
브랜치나 저장소가 목록에 없어요 21
마지막 한 장: 작업 체크리스트 21

팀 프로젝트를 내 컴퓨터로 가져오고, 내 브랜치에서 작업하는 방법을 안내합니다.

Unity 에서 저장한 변경을 확인하고, Commit 다음 Push 까지 마칠 수 있습니다.

기준 화면은 **Windows 용 GitHub Desktop 3.6.10** 입니다. 그림은 격리된 데모 저장소에서 촬영했습니다. 파일과 작업 내용은 설명용 예시입니다.

한눈에 보기

순서	할 일	끝났는지 확인할 곳
처음 한 번	설치 → 로그인 → Clone	Current repository 에 CK_Semester_project 표시
매일 시작	Fetch origin → 필요하면 Pull origin	새 변경을 받은 뒤 작업 시작
작업 공간	기준 브랜치 확인 → 내 브랜치 선택 또는 생성	Current branch 의 이름
Unity 작업	작업 → Unity 에서 저장 → Unity 닫기	문서 2 의 저장 단계 참고
내 컴퓨터에 기록	Changes 확인 → Summary 작성 → Commit	History 에 내 기록 표시
팀에 전달	첫 번째는 Publish branch, 이후에는 Push origin	전송 완료 후 Fetch origin 으로 돌아옴

> 팁 **Commit 다음 Push 까지 해야 끝입니다.** Commit 은 내 컴퓨터의 기록입니다. GitHub 에도 올라가려면 Push 가 필요합니다. Unity 의 저장도 먼저 해야 합니다.

1. 브랜치와 저장의 차이

작업공간을 나눠서 따로 작업한 뒤 main 에 합치는 방식입니다. 이 작업공간을 브랜치(Branch)라고 합니다. 공용 원본을 바로 고치는 대신 내 브랜치에서 수정하고, 프로그래머가 필요한 내용을 합칩니다.

브랜치마다 프로젝트 폴더를 따로 만드는 것은 아닙니다. GitHub Desktop 에서 브랜치를 바꾸면 같은 폴더의 파일 내용이 그 브랜치에 맞게 바뀝니다. 그래서 Unity 를 켠 채 브랜치를 옮기면 헷갈리거나 파일이 다시 바뀔 수 있습니다.

화면 용어	쉬운 뜻	기억할 점

화면 용어	쉬운 뜻	기억할 점
Repository, 저장소	프로젝트 파일과 변경 기록을 담는 곳	이 팀은 CK_Semester_project 사용
Branch, 브랜치	작업을 나누는 갈래	Current branch 가 지금 작업할 곳
Commit, 커밋	선택한 변경을 내 컴퓨터에 한 묶음으로 기록	GitHub 전송은 아직 안 됨
Push	내 커밋을 원격 저장소로 전송	팀원이 받으려면 여기까지 필요
Publish branch	새 브랜치를 원격에 처음 올리기	다음부터는 Push origin 사용
Fetch	원격에 새 기록이 있는지 확인하고 그 정보를 가져오기	작업 중인 파일에 새 내용을 적용하지는 않음
Pull	원격의 새 기록을 현재 브랜치에 적용	Unity 를 닫고 받는 것이 안전
Stash	아직 커밋하지 않은 변경을 잠시 보관	내 컴퓨터의 임시 보관이며 Push 되지 않음
Restore	임시 보관한 변경을 Changes 로 꺼내기	꺼낸 뒤에도 Commit 과 Push 필요
origin	이 저장소에 연결된 원격의 이름	실제 팀 작업에서는 GitHub 의 팀 저장소

화면에서 먼저 볼 여섯 곳

1. **Current repository**: 지금 연 저장소입니다. `CK_Semester_project` 인지 확인합니다.
2. **Current branch**: 지금 작업할 브랜치입니다. 작업 전과 Commit 전에 확인합니다.
3. **Fetch origin**: 원격의 새 기록을 확인합니다. 상황에 따라 Pull origin, Push origin, Publish branch 로 바뀝니다.
4. **Changes**: 아직 커밋하지 않은 파일 변경입니다.
5. **History**: 이미 커밋한 기록입니다.
6. **Summary (required)** 입력 칸: 이번 커밋의 제목을 적습니다.

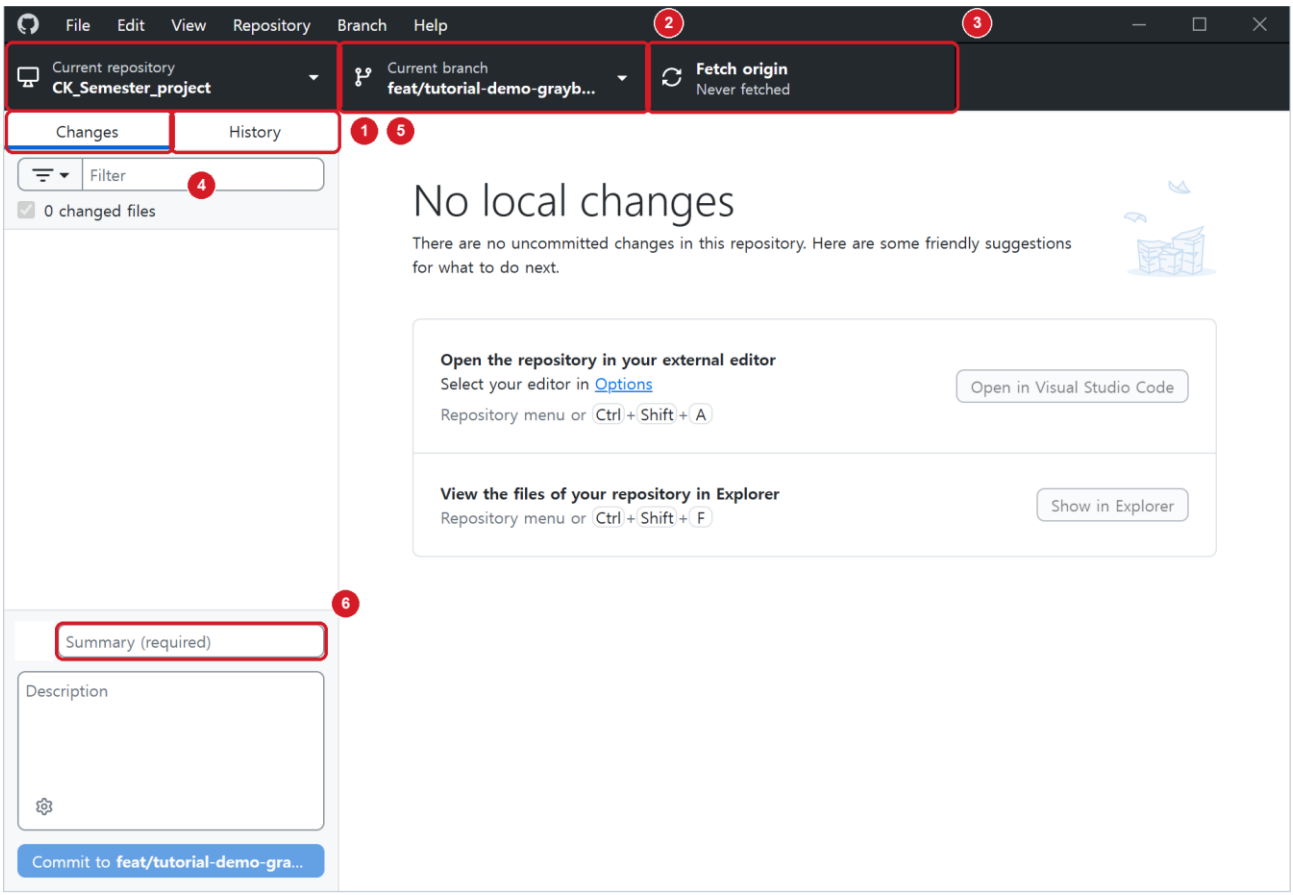


그림 1. 화면의 여섯 영역: 저장소, 브랜치, 동기화 버튼, Changes, History, 커밋 제목

2. 처음 한 번: 설치·로그인·Clone

설치하고 로그인하기

공식 배포 주소 `https://desktop.github.com/`에서 Windows 용 GitHub Desktop 을 설치합니다. 설치 파일의 안내를 마치고 앱을 실행합니다.

1. 시작 화면의 **Sign in to GitHub.com** 을 누릅니다.

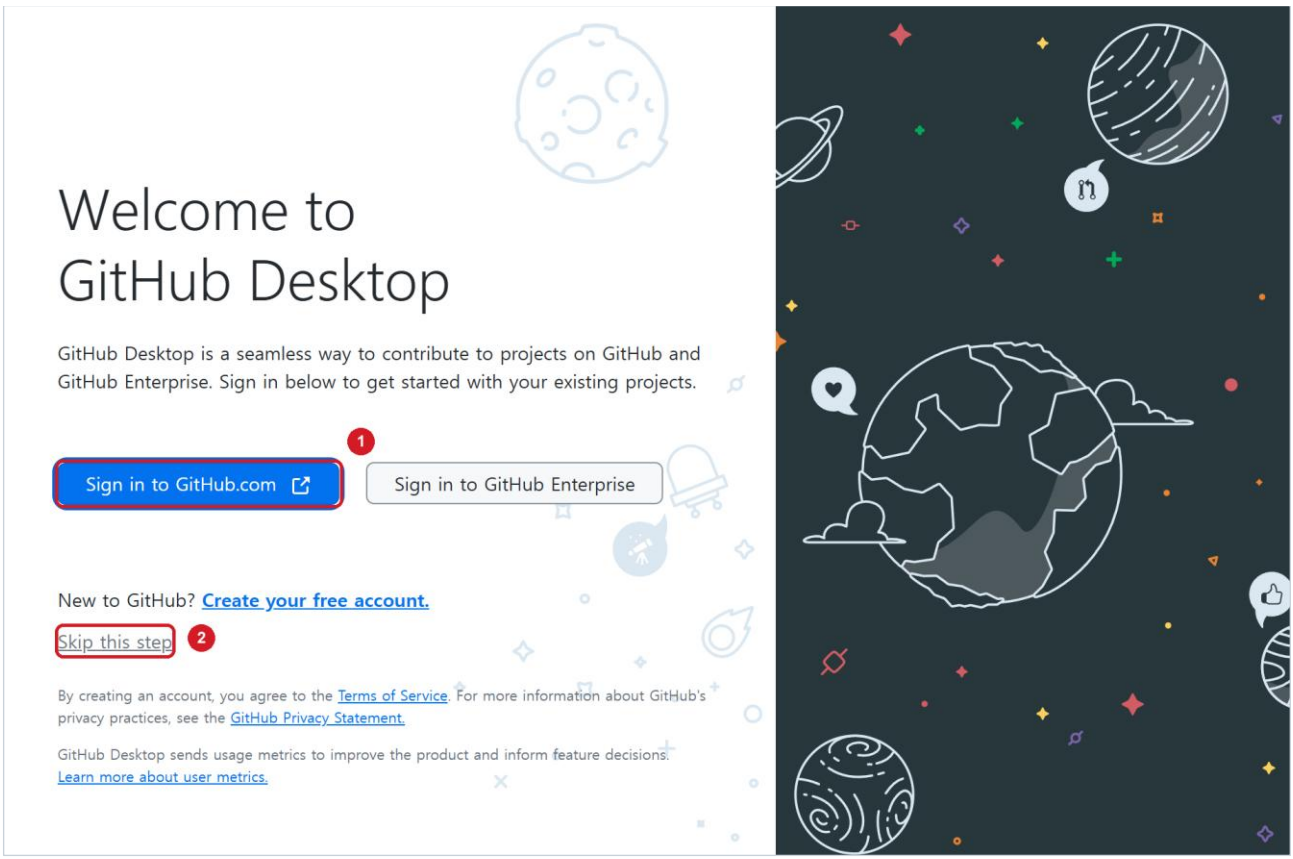


그림 2. 첫 실행 화면. 1 은 로그인, 2 는 이 단계를 건너뛰는 버튼

브라우저에 열린 안내에 따라 본인의 GitHub 계정으로 로그인합니다. 팀 저장소 접근 권한이 있는 계정이어야 합니다. GitHub Desktop 으로 돌아오면 안내에 따라 Git 작성자 설정을 확인하고 마칩니다.

> **참고** 그림의 2 번 **Skip this step** 은 로그인을 건너뛰는 버튼입니다. 실제 팀 저장소를 받고 전송하려면 팀 권한이 있는 계정으로 로그인하세요. 이미 설정을 끝낸 사람은 다음 단계로 갑니다.

팀 저장소 가져오기

Clone 은 GitHub 에 있는 저장소를 내 컴퓨터에 처음 내려받는 일입니다. 이미 받은 프로젝트라면 다시 Clone 하지 말고 기존 저장소를 엽니다.

상단 **File** 메뉴에서 **Clone repository...**를 엽니다. 아래 그림의 대화상자가 나오면 순서대로 진행합니다.

1. **URL** 탭을 누릅니다.
2. **Repository URL or GitHub username and repository** 입력 칸에 `https://github.com/dunfr/CK_Semester_project` 를 입력합니다.
3. **Local path** 에 내 컴퓨터의 저장 위치를 정합니다. 예: `D:\TeamProject\CK_Semester_project`.
4. 주소와 저장 위치를 확인한 뒤 **Clone** 을 누릅니다.

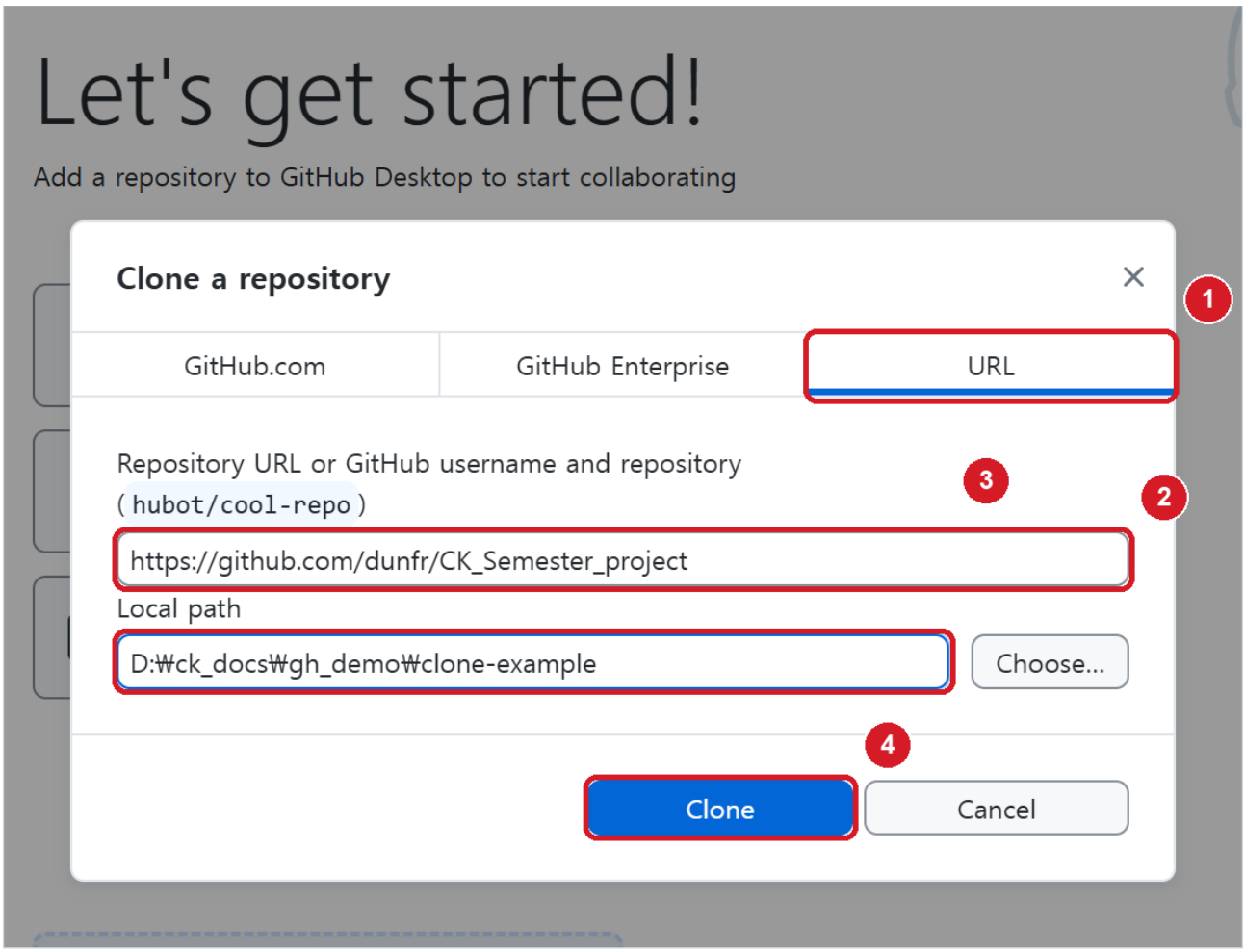


그림 3. Clone 대화상자: URL 탭, 팀 저장소 주소, 저장 위치, Clone 버튼

> **주의** 그림의 Local path 는 설명용 경로입니다. 본인의 프로젝트 저장 위치를 정하세요. 이미 파일이 있는 프로젝트 폴더를 덮어쓰지 마세요.

완료되면 Current repository 에 `CK_Semester_project` 가 보입니다. Unity 에서 여는 프로젝트 폴더도 이 저장소 폴더와 같아야 합니다. 프로젝트 열기와 저장은 **문서 2 의 해당 단계**를 따라 합니다.

3. 매일 시작: 새 변경부터 받기

Unity 가 열려 있다면 먼저 저장하고 닫습니다. 저장 방법은 **문서 2 의 Unity 저장 단계**를 참고하세요.

1. GitHub Desktop 상단의 **Fetch origin** 을 누릅니다.

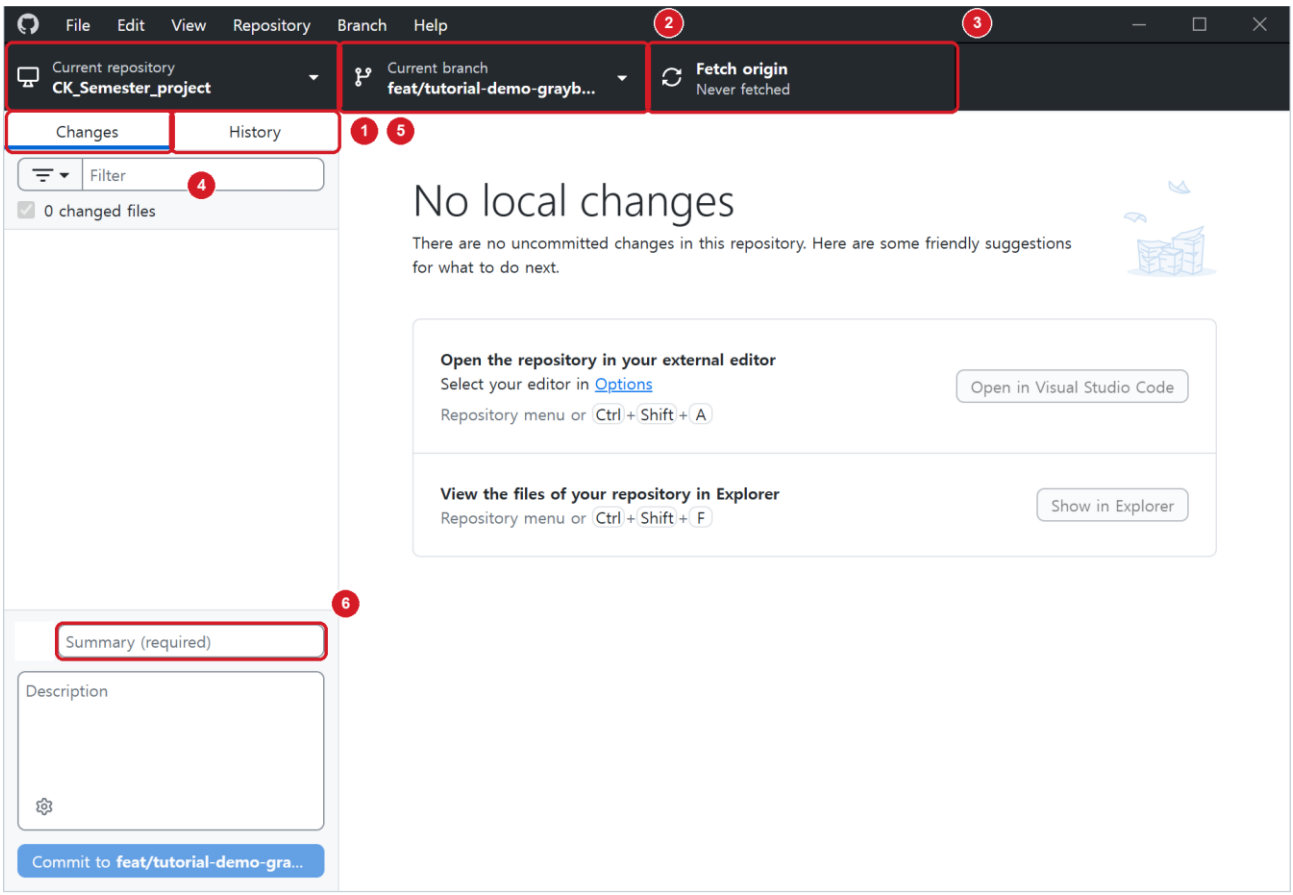


그림 4. 시작할 때 확인하는 Fetch origin 은 3 번

Fetch 가 끝났을 때 **Pull origin** 이 보이면 아래 단계를 진행합니다. Fetch origin 이 그대로라면 현재 브랜치에 받을 새 커밋이 없는 상태입니다.

2. **Pull origin** 을 누릅니다.

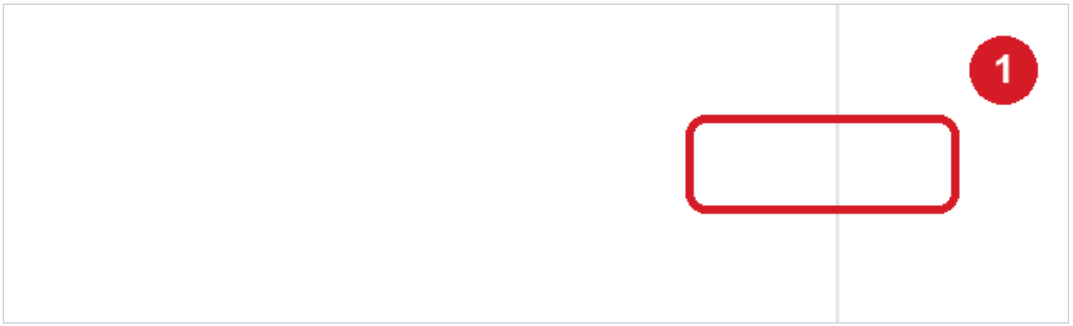


그림 5. 새 기록이 있을 때 나타나는 Pull origin 버튼

Pull 이 끝날 때까지 기다립니다. 오류나 충돌 안내가 나오면 Unity 작업을 시작하지 말고 프로그래머에게 알립니다.

> **참고** Fetch 와 Pull 은 선택한 브랜치에 대한 확인입니다. 이어서 다른 브랜치를 골랐다면 그 브랜치에서도 Fetch/Pull 을 확인하세요.

4. 작업할 브랜치 고르기

주요 브랜치

작업	브랜치 이름	사용 방법
캐릭터·배경 쉐이더 조정	feat/shader-tweaks	담당자가 한 명만 작업하기로 약속했다면 이 브랜치에서 작업 가능
모든 기능을 통합 적용 중인 프로토타입	feat/tutorial-demo-grayboxing	프로토타입을 기준으로 새 작업을 시작할 때 프로그래머와 기준 확인
맵 그레이박싱	feat/map-grayboxing-fuchsia	맵 작업의 기준을 확인하고 개인 브랜치를 만들기
통합 대상	main	그래픽 파트가 직접 커밋·전송하거나 합치지 않음

기존 작업을 이어 한다면 **전에 사용한 내 브랜치**로 돌아갑니다. 새 브랜치를 매일 만들 필요는 없습니다.

기존 브랜치로 이동하기

상단 **Current branch** 를 눌러 목록을 엽니다.

1. 쉐이더 조정은 `feat/shader-tweaks` 입니다.
2. 통합 프로토타입은 `feat/tutorial-demo-grayboxing` 입니다.
3. 맵 그레이박싱은 `feat/map-grayboxing-fuchsia` 입니다.
4. 내 브랜치를 새로 만들 때는 **New branch** 를 누릅니다.

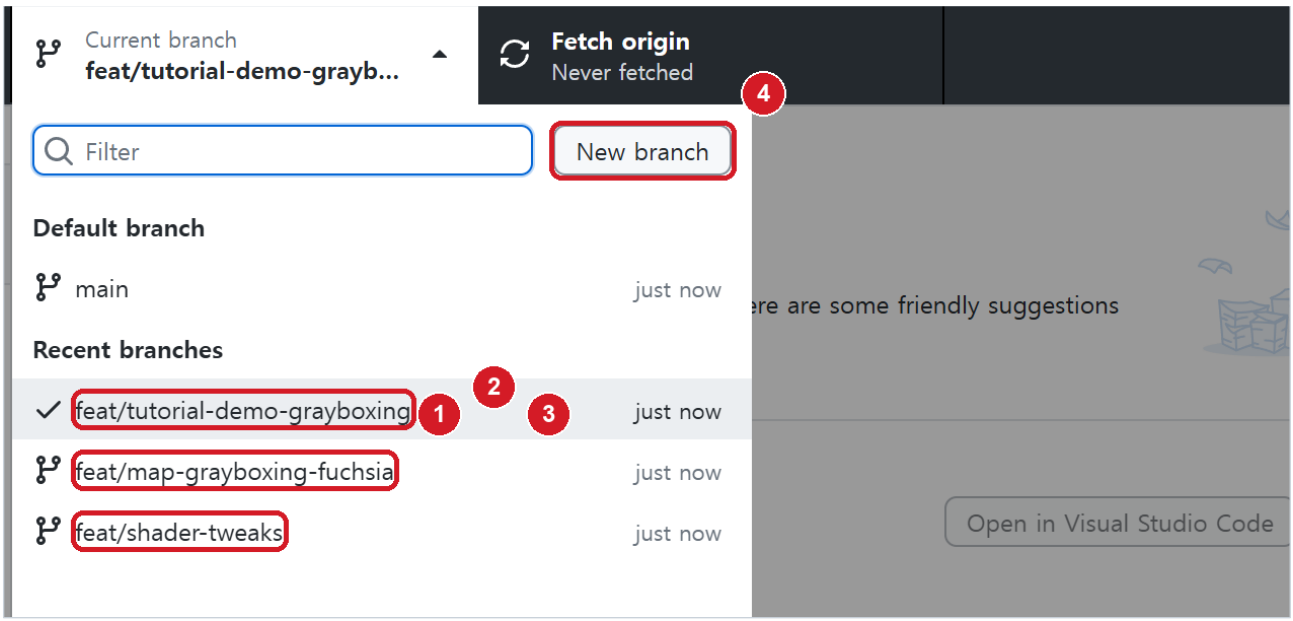


그림 6. 주요 브랜치 세 개와 New branch 버튼

목록에서 원하는 브랜치 이름을 누르면 이동합니다. 상단 Current branch 의 이름이 바뀌었는지 확인합니다. 목록에서 찾기 어렵다면 검색 칸에 브랜치 이름을 입력합니다.

> **주의** 한 브랜치에 두 명이 동시에 작업하면 서로의 내용을 덮어쓰거나 충돌을 해결하다 작업을 잃을 수 있습니다. **자기 브랜치를 만들어 작업하는 것을 권장합니다.** 쉐이더 조정처럼 같은 브랜치를 써야 한다면 한 명만 작업한다는 약속과 인계가 필요합니다.

5. 새 브랜치 만들기

어느 브랜치에서 시작하나요?

새 브랜치는 **시작할 때 선택한 기준의 기록**을 이어받습니다. 브랜치를 만들었다고 다른 브랜치의 최신 작업이 자동으로 계속 들어오지는 않습니다.

프로그래머에게 이번 작업의 기준 브랜치를 먼저 확인합니다. 그 브랜치로 이동하고 Fetch/Pull 을 마친 다음 만듭니다. 무조건 `main` 에서 시작하면 팀의 진행 중인 씬이나 기능이 빠질 수 있습니다.

여기서는 `feat/tutorial-demo-grayboxing` 을 기준으로 복도 색 조정용 브랜치를 만드는 예시입니다. 맵 작업이라면 같은 방식으로 맵 기준 브랜치에서 시작합니다.

브랜치 이름은 **작업 종류/구분되는 작업 이름**으로 짧게 적습니다. 팀에서 정한 이름이 있으면 그 이름을 우선합니다. 아래는 설명용 예시이며, 별도로 확인된 팀의 강제 이름 규칙은 아닙니다.

예시 이름	뜻
feat/art-corridor-color	복도 색 조정 작업

예시 이름	뜻
feat/art-map-lobby	로비 맵 작업
fix/art-wall-material	벽 재질 수정

이름이 겹치면 작업 구역이나 팀에서 합의한 담당자 구분을 붙입니다. 공백 대신 `-`를 사용합니다.

생성하고 이름 확인하기

기준 브랜치를 선택한 상태에서 Current branch 를 열고 **New branch** 를 누릅니다. 새 브랜치 창에서 진행합니다.

1. **Name** 에 `feat/art-corridor-color` 처럼 내 작업 이름을 입력합니다.
2. **Create branch based on...** 선택이 나오면 합의한 기준 브랜치를 고릅니다. 예시는 `feat/tutorial-demo-grayboxing` 입니다.
3. **Create branch** 를 누릅니다.

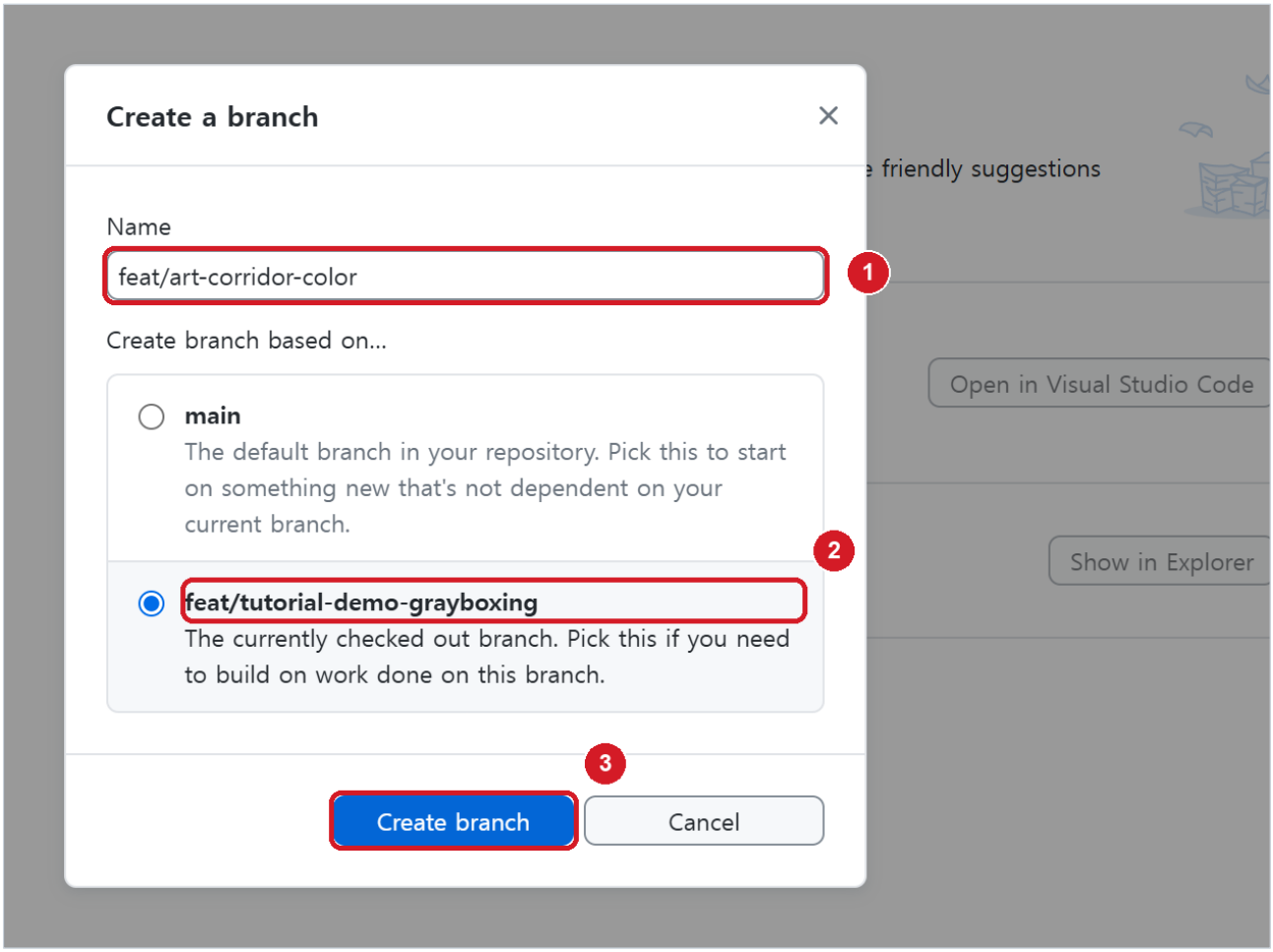


그림 7. 새 브랜치 이름, 기준 브랜치 선택, Create branch 버튼

기준 선택이 없는 경우에는 창에 표시된 기준 브랜치 설명을 읽습니다. 합의한 기준과 다르면 Cancel 로 닫고 올바른 브랜치로 이동한 뒤 다시 만듭니다.

새 브랜치 처음 올리기

1. **Current branch** 가 방금 만든 내 브랜치인지 확인합니다.
2. **Publish branch** 를 누릅니다.

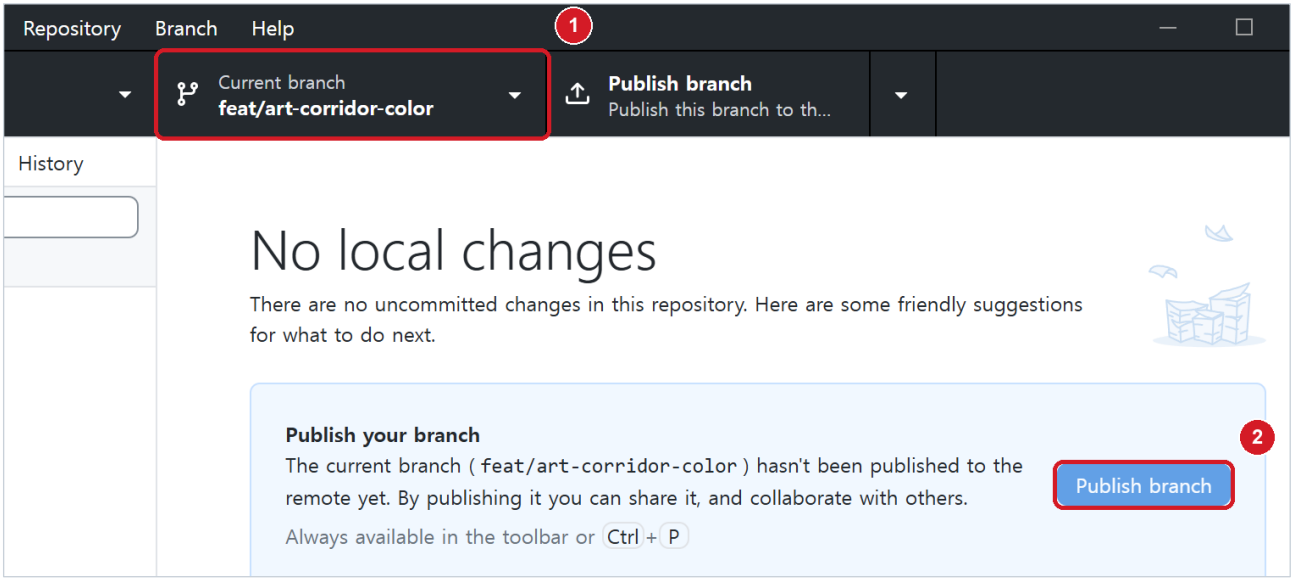


그림 8. 새 브랜치 이름 확인과 첫 전송인 Publish branch

Publish branch 는 새 브랜치를 GitHub 에 처음 만드는 전송입니다. 이때 아직 하지 않은 Unity 작업까지 저장되는 것은 아닙니다. 이후의 변경은 Unity 저장 → Commit → Push origin 순서로 보냅니다.

> **참고** Publish branch 를 작업의 첫 커밋 뒤에 눌러도 됩니다. 그때는 새 브랜치와 이미 만든 커밋을 함께 전송합니다. 첫 전송 후에는 Push origin 을 사용합니다.

6. Unity 에서 작업하고 저장하기

Unity 에서 **Clone 한 프로젝트 폴더**를 엽니다. 합의한 씬·재질·구역에서 작업합니다. 다른 사람과 같은 씬이나 프리팹을 동시에 수정하지 않도록 먼저 나눕니다.

작업이 끝나면 **문서 2 의 Unity 저장 단계**를 따라 저장합니다. 씬의 변경과 프로젝트 파일의 저장 여부를 확인한 뒤 Unity 를 닫습니다. GitHub Desktop 으로 돌아옵니다.

1. **Changes** 에서 바뀐 파일을 확인합니다. 예시의 1 번 `.mat` 은 재질, 2 번 `.unity` 는 씬 파일입니다.
2. 예시의 3 번은 Unity 가 자동으로 바꾼 설정 파일입니다. 다음 절에서 커밋 대상에서 뺍니다.

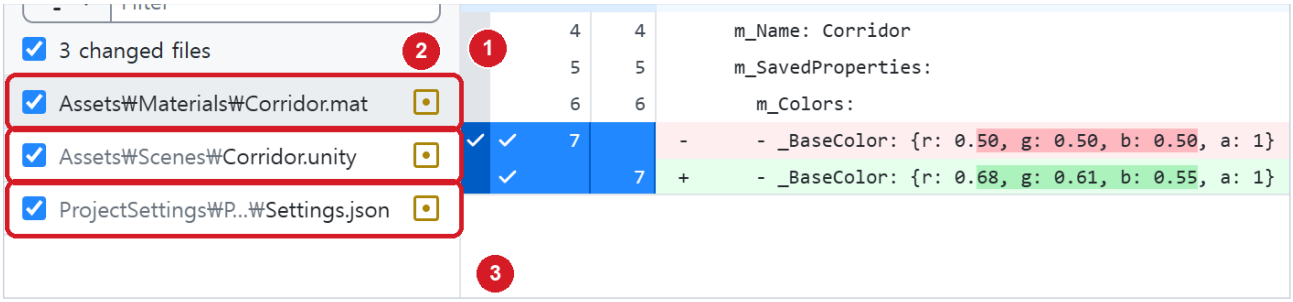


그림 9. Unity 저장 후 나타난 재질, 씬, 자동 변경 설정 파일과 체크 상태

파일을 누르면 오른쪽에 변경 내용이 보입니다. 모르는 파일이 많거나 본인이 수정하지 않은 씬이 나타나면 Commit 전에 프로그래머에게 확인합니다.

> **참고** 저장은 Unity 에서, 변경 확인은 GitHub Desktop 에서 합니다. Unity 화면과 저장 방법은 문서 2 를 참고하세요. Changes 가 비어 있어도 아직 저장하지 않은 Unity 작업은 남아 있을 수 있습니다.

7. 커밋할 파일 고르기

체크된 파일의 선택된 변경만 이번 커밋에 들어갑니다. **체크 해제는 파일을 지우거나 원래대로 되돌리는 동작이 아닙니다.** 제외한 변경은 Changes 에 계속 남습니다.

1. 예시의 재질 `.mat` 파일은 체크를 유지합니다.
2. 예시의 씬 `.unity` 파일도 이번 작업에 필요한지 확인하고 체크를 유지합니다.
3. **`ProjectSettings/Packages/com.unity.probuilder/Settings.json`은 체크를 해제합니다.** Unity 가 혼자 바꾸는 이 파일은 이번 팀 작업에서 커밋하지 않습니다.

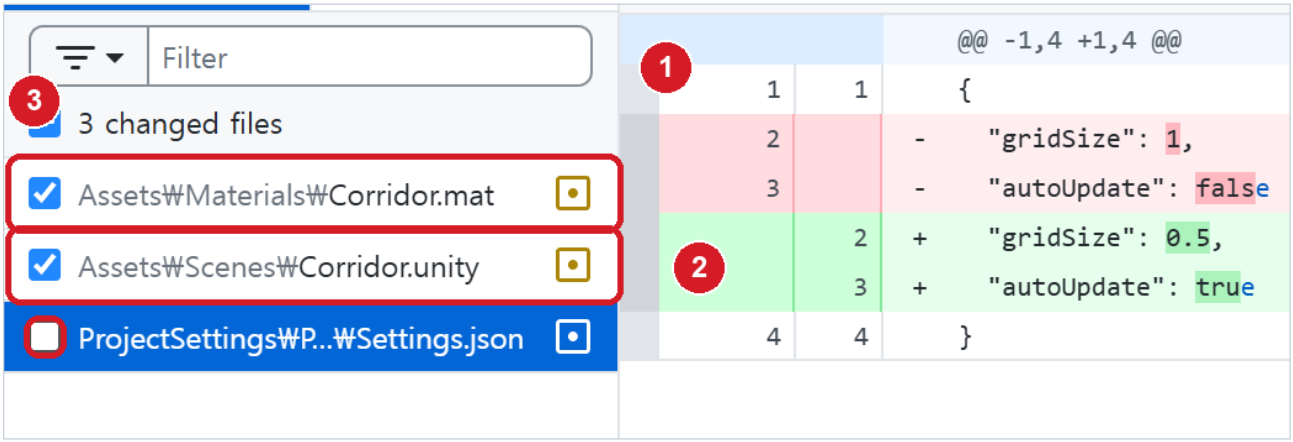


그림 10. 필요한 파일은 체크하고 자동 변경 Settings.json 은 체크 해제한 상태

함께 추가한 에셋의 `.meta` 파일은 Unity 가 에셋을 구분하는 번호를 담습니다. 필요한 에셋과 해당 `.meta` 를 함께 커밋합니다. `.meta` 를 임의로 지우거나 이름만 따로 바꾸지 마세요.

> **주의** 제외한 Settings.json 변경이 남아 있으면 나중에 브랜치를 바꿀 때 안내가 나올 수 있습니다. 다른 작업과 섞어 버리지 마세요. 본인이 한 변경인지 확실하지 않으면 되돌리지 말고 프로그래머에게 확인하세요.

8. Summary 작성하고 Commit 하기

팀 커밋 제목 규칙

제목은 `**종류 : 무엇을 바꿨는지**` 형식으로 적습니다. 팀의 Git 사용 표준과 실제 커밋에 쓰인 형식입니다. 예: `feat : 복도 재질 색 조정`.

종류	팀에서 사용하는 뜻	제목 예시
feat	기능이나 작업 내용 추가	feat : 복도 재질 색 조정
fix	잘못된 내용 수정	fix : 벽 재질 누락 수정
docs	문서 내용 수정	docs : 맵 작업 안내 수정
style	코드·기능에 영향 없는 변경	style : 프리팹 모델링 변경
refactor	구조 정리, 리팩토링	refactor : 기존 맵 생성 시스템 정리
test	테스트용 작업	test : 복도 조명 테스트 씬 추가
chore	빌드·프로젝트 설정·패키지 등	chore : 프로젝트 씬 설정 변경

처음에는 한 번의 목적을 한 커밋으로 묶습니다. `수정`, `저장`, `완료`만 적기보다 어느 씬·재질을 어떻게 바꿨는지 적습니다. 여러 내용을 설명해야 하면 아래 설명 칸에 적습니다.

내 컴퓨터에 기록하기

Commit 전에 Current branch 가 내 작업 브랜치인지, 체크한 파일이 맞는지 한 번 더 확인합니다.

1. **Summary (required)** 입력 칸에 `feat : 복도 재질 색 조정`처럼 제목을 적습니다. 입력하면 안내 문구 대신 적은 제목이 보입니다.
2. 예시의 **Commit 2 files to feat/art-corridor-...** 버튼을 누릅니다. 선택된 파일 수와 내 브랜치가 맞는지 먼저 확인합니다. 브랜치 이름이 길면 버튼에서 끝부분이 생략됩니다. 전체 이름은 상단 Current branch 에서 확인합니다.

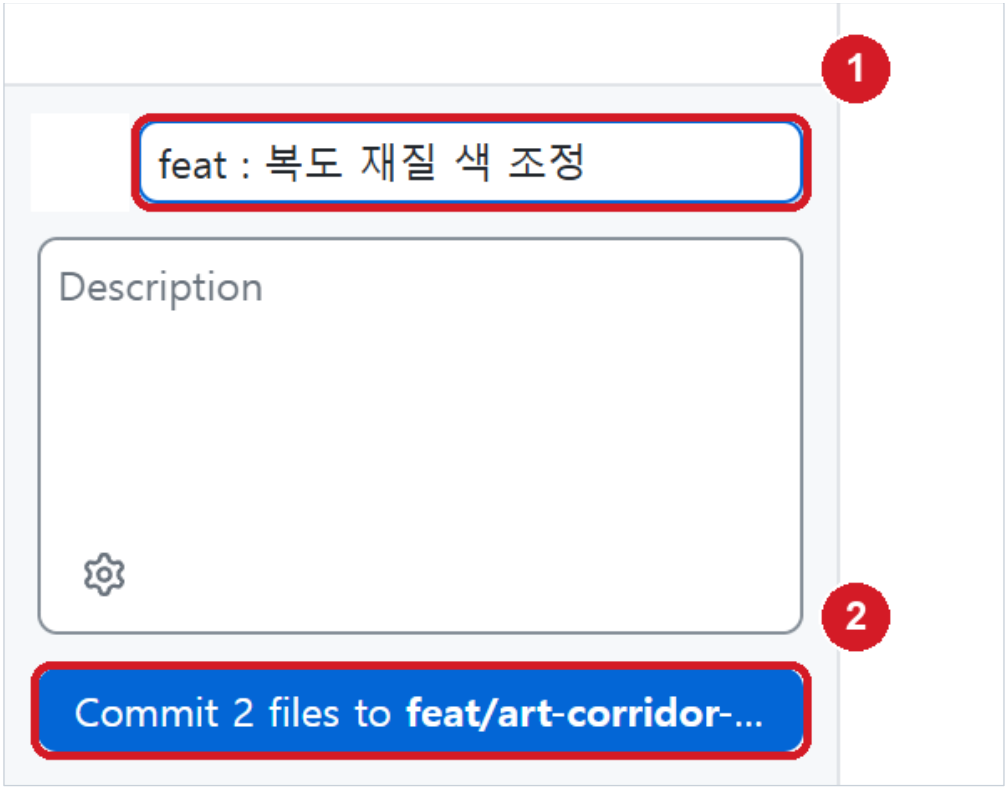


그림 11. Summary 입력과 현재 브랜치에 커밋하는 버튼

기록이 끝나면 커밋한 파일은 Changes 에서 사라집니다. 체크를 해제한 파일은 남을 수 있습니다. **이 단계는 내 컴퓨터에만 기록한 상태입니다.**

9. Push 까지 마치고 확인하기

1. 상단의 **Push origin** 을 누릅니다. 새 브랜치를 아직 전송한 적이 없다면 **Publish branch** 를 누릅니다.

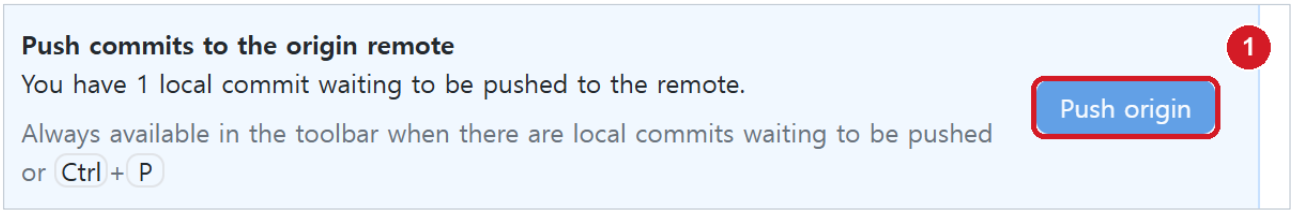


그림 12. 기록한 변경을 원격으로 보내는 Push origin

전송이 끝날 때까지 기다립니다. 오류 없이 끝나고 버튼이 Fetch origin 으로 돌아왔는지 확인합니다.

2. **Fetch origin** 으로 돌아온 것을 확인합니다.
3. 예시처럼 **No local changes** 가 나오면 아직 커밋하지 않은 파일 변경도 없는 상태입니다.

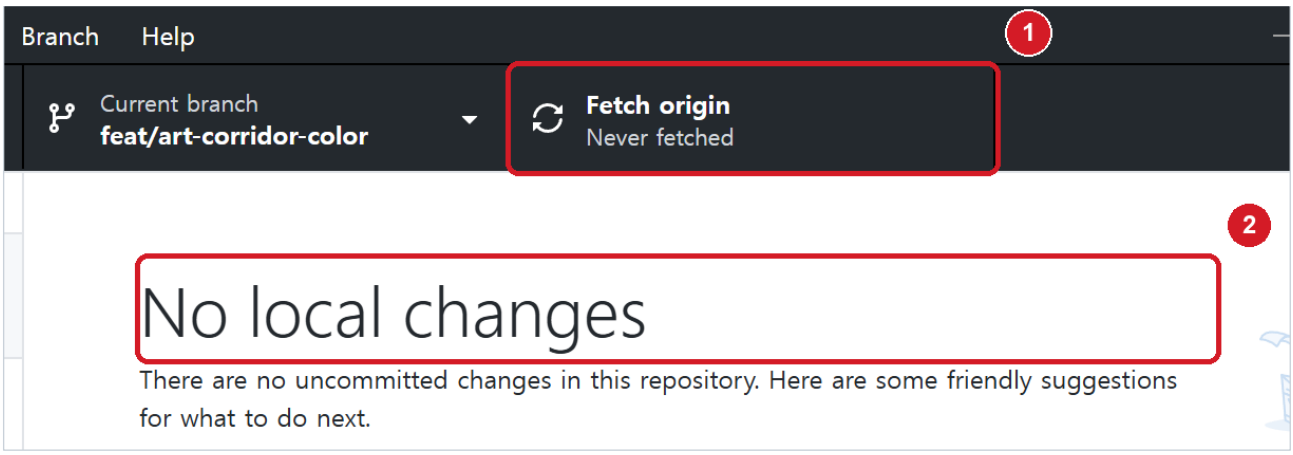


그림 13. 전송 후 Fetch origin 과 No local changes 가 보이는 예시

> 참고 **No local changes 만으로 Push 완료를 판단하지 마세요.** Commit 만 해도 이 문구가 나올 수 있습니다. Push 또는 Publish branch 가 오류 없이 끝났는지 함께 확인합니다. 의도적으로 제외한 설정 변경이 있으면 No local changes 는 나오지 않을 수 있습니다.

기록을 다시 확인하려면 다음과 같이 합니다.

4. **History** 탭을 누릅니다.
5. 방금 적은 커밋 제목이 있는지 확인합니다.

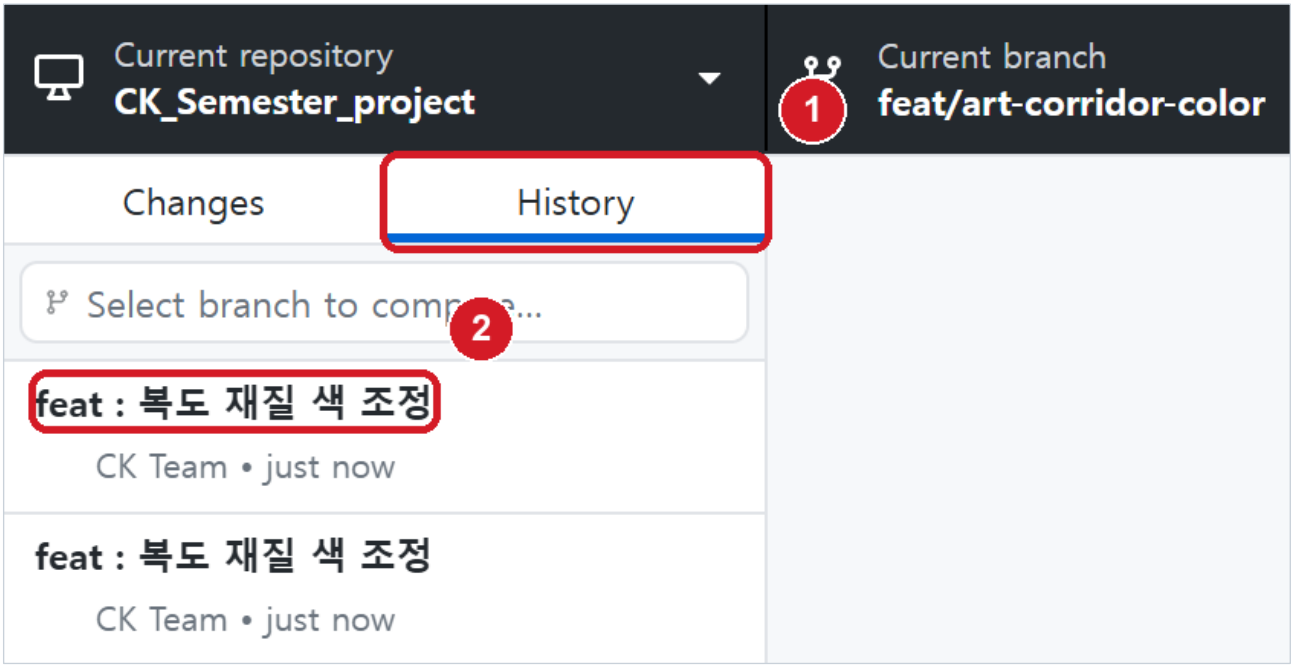


그림 14. History 탭과 방금 만든 커밋 제목

> 참고 History 는 내 컴퓨터의 커밋도 보여 줍니다. History 에 제목이 있다는 사실만으로 GitHub 에 전송된 것은 아닙니다. 항상 **Commit 다음 Push 까지** 마치세요.

작업을 합칠 때는 프로그래머에게 **브랜치 이름, 커밋 제목, 바꾼 씬·재질, 확인할 점**을 전달합니다. 그래픽 파트가 main 으로 직접 합치지 않습니다.

10. 브랜치를 옮길 때 변경이 남아 있다면

브랜치를 옮기기 전에 Unity 에서 저장하고 닫습니다. Changes 에 커밋하지 않은 변경이 있으면 아래와 같은 선택이 나올 수 있습니다. 무심코 확인하지 말고 변경을 어디에 둘지 고릅니다.

1. **Leave my changes on 현재 브랜치**: 이 브랜치에 변경을 임시 보관합니다. 나중에 돌아와 꺼낼 때 씁니다.
2. **Bring my changes to 이동할 브랜치**: 변경을 새 브랜치로 가져갑니다. 잘못된 브랜치에서 아직 커밋하지 않은 작업을 옮길 때 씁니다.
3. 선택한 동작과 목적지를 확인한 뒤 **Switch branch** 를 누릅니다.

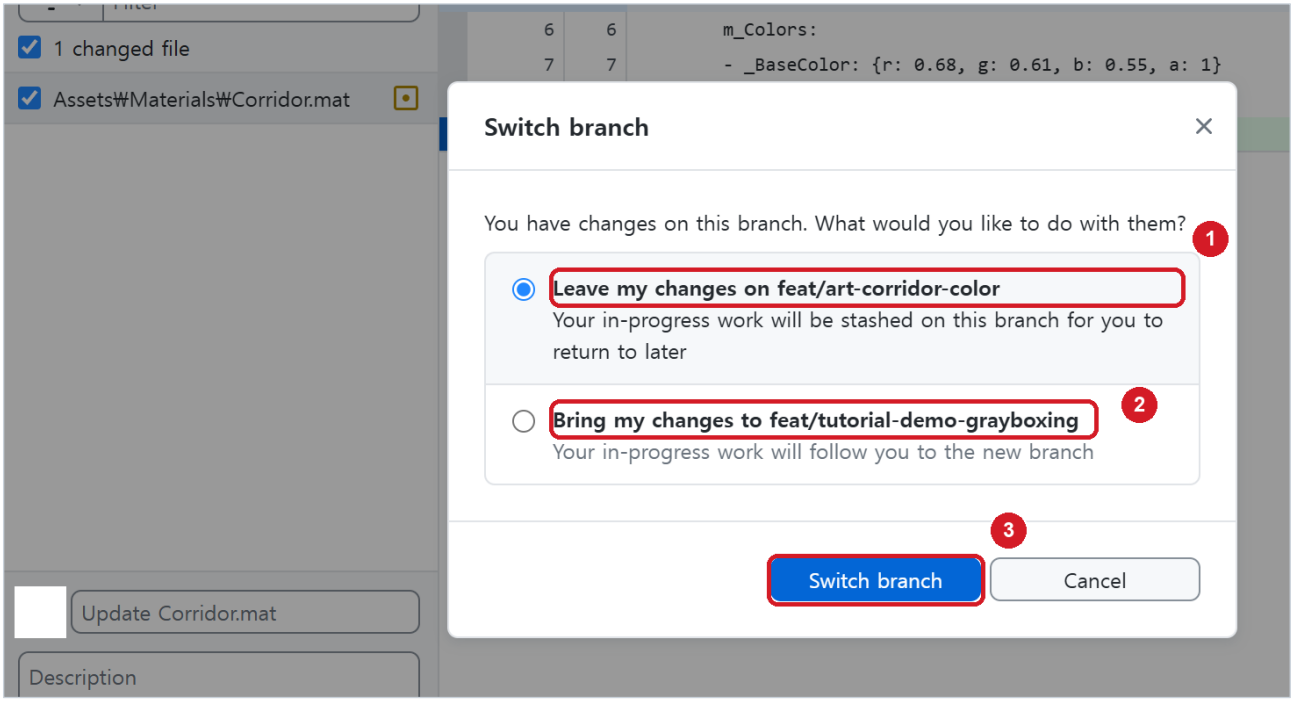


그림 15. 브랜치에 변경을 두고 갈지, 이동할 브랜치로 가져갈지 고르는 창

> **주의** 기존 Stash 를 덮어쓴다는 경고가 나오면 진행하지 마세요. 임시 보관한 작업부터 확인합니다. 확실하지 않으면 Cancel 로 돌아가 프로그래머에게 요청합니다.

사라진 것처럼 보이는 작업 꺼내기

Leave my changes 를 골랐다면 원래 브랜치로 돌아옵니다. 해당 브랜치의 Changes 화면에서 진행합니다.

1. 왼쪽의 **Stashed Changes** 를 누릅니다. 화면에 **View stash** 가 보이면 그 버튼으로 열어도 됩니다.
2. 보관된 파일이 내 작업인지 확인하고 **Restore** 를 누릅니다.

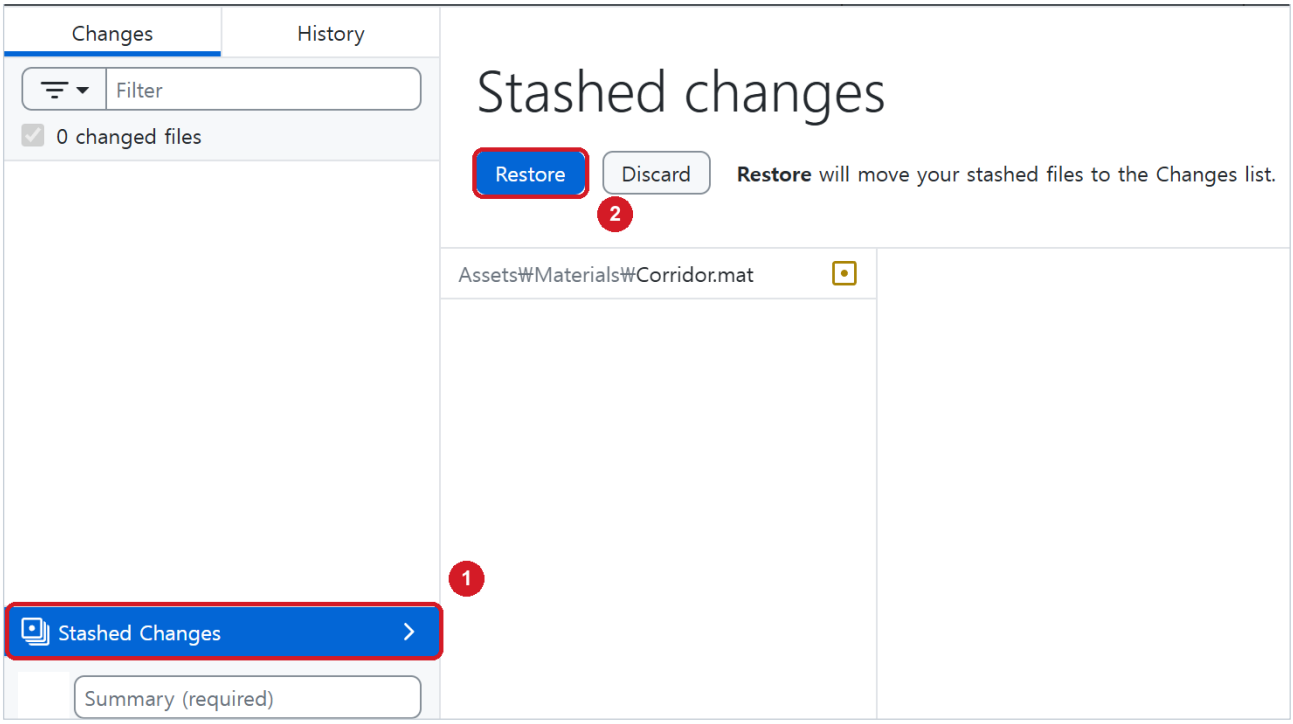


그림 16. Stashed Changes 와 보관된 변경을 다시 꺼내는 Restore 버튼

꺼낸 뒤에는 다음 상태를 확인합니다.

3. **Changes** 탭을 엽니다.

4. 내 파일 변경이 목록에 돌아왔는지 확인합니다.

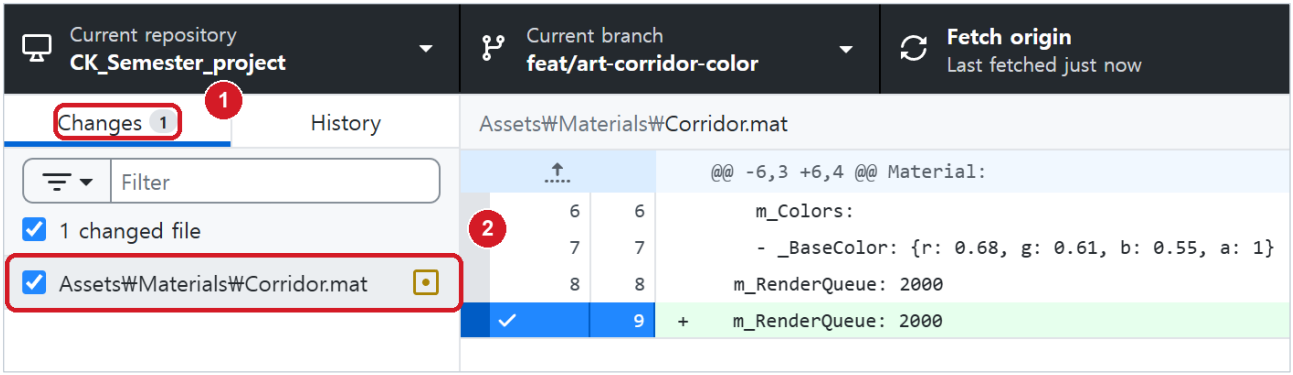


그림 17. Restore 후 재질 변경이 Changes 목록으로 돌아온 상태

Restore 는 커밋이나 전송이 아닙니다. 계속 작업하거나, 변경을 확인하고 Commit → Push 를 마칩니다. Stash 는 PC 를 옮겨 가는 백업으로 쓰지 마세요.

11. 꼭 지킬 주의사항

- **main 에 직접 올리지 않습니다.** 다른 브랜치와 합치는 일도 프로그래머에게 요청합니다.
- 한 브랜치는 한 명이 맡습니다. 같은 브랜치의 담당자를 바꿀 때는 Commit·Push 완료와 작업 구역을 인계합니다.
- Unity 의 씬 `.unity` 와 프리팹 `.prefab` 은 같은 부분을 동시에 바꾸면 합치기 어렵습니다. 개인 브랜치라도 같은 씬을 동시에 고치는 문제는 남습니다. 씬·구역·담당을 먼저 나눕니다.

- 브랜치를 바꾸거나 Pull 하기 전에 Unity 에서 저장하고 닫습니다. Unity 저장 방법은 문서 2 를 참고합니다.
- `ProjectSettings/Packages/com.unity.probuilder/Settings.json` 은 커밋 대상에서 제외합니다.
- 이 저장소에는 Git LFS 가 없습니다. Git LFS 는 큰 파일을 별도로 관리하는 기능입니다. 큰 원본 파일을 임의로 커밋하지 않습니다.
- GitHub 는 일반 Git 파일 **100 MiB 초과**를 거부합니다. 흔히 100MB 제한이라고 부릅니다. 50 MiB 초과도 경고 대상이므로 추가 전에 프로그래머에게 크기와 전달 방법을 확인합니다.
- 파일의 **Discard changes** 는 체크 해제와 다릅니다. 아직 커밋하지 않은 수정을 버리는 동작입니다. 자동 변경임이 확실한 해당 파일만 확인하고 처리하며, 모르는 씬·재질에는 사용하지 않습니다.
- Force push, Reset, Rebase, 충돌 해결은 혼자 진행하지 않습니다. 오류가 나면 현재 상태를 보존하고 프로그래머에게 요청합니다.

12. 자주 겪는 문제

브랜치를 옮겼더니 작업이 사라졌어요

원래 브랜치로 돌아갑니다. Stashed Changes 또는 View stash 를 열고 내용을 확인한 뒤 Restore 를 누릅니다. 위의 임시 보관 복구 그림을 따라 하세요. Stash 가 없다면 다른 브랜치의 History 와 Unity 저장 여부를 프로그래머와 함께 확인합니다.

Changes 에 아무것도 안 떠요

Unity 에서 아직 저장하지 않았거나 다른 프로젝트 폴더를 열었을 수 있습니다. 문서 2 의 저장 단계를 따라 저장하고, GitHub Desktop 의 Current repository 와 Unity 프로젝트 폴더가 같은지 확인합니다. 이미 Commit 했다면 Changes 대신 History 를 봅니다.

Push 가 안 돼요

상대가 먼저 올린 커밋 때문에 거부됐다면 Unity 를 닫고 Fetch origin 을 누릅니다. Pull origin 이 나오면 받은 뒤 다시 Push 합니다. Pull 도중 충돌이 나면 중단하고 프로그래머에게 알립니다.

로그인·권한·네트워크 오류는 Pull 만으로 해결되지 않습니다. 메시지를 읽고 팀 계정의 접근 권한과 네트워크 연결을 확인합니다. Force push 로 밀어붙이지 마세요.

충돌이 났어요

충돌은 같은 부분의 두 변경 중 어느 쪽을 남길지 Git 이 결정하지 못한 상태입니다. 특히 씬·프리팹에서는 글자 몇 줄만 보고 고르면 오브젝트 설정을 잃을 수 있습니다. **혼자 해결하지 말고 프로그래머를 호출합니다.**

현재 브랜치, 실행한 동작, 충돌 파일 이름을 전달합니다. 임의로 한쪽 버전을 선택하거나 Commit·Push·Discard 를 반복하지 않습니다.

대용량 파일 오류가 나요

파일 하나의 크기를 확인합니다. 커밋하기 전이면 해당 파일의 체크를 해제하고 프로그래머에게 전달 방법을 묻습니다. **이미 커밋했다면 파일을 지우고 새 커밋을 하나 더 만드는 것만으로는 해결되지 않을 수 있습니다.** 이전 기록에 큰 파일이 남으므로 프로그래머에게 커밋 정리를 요청합니다.

실수로 잘못된 브랜치에 작업했어요

아직 Commit 전이면 Unity 를 저장하고 닫습니다. 올바른 개인 브랜치로 옮길 때 Bring my changes to 를 선택합니다. 목적지 이름과 변경 목록을 확인합니다. 이동 중 경고나 충돌이 나오면 Cancel 로 돌아가 도움을 요청합니다.

이미 Commit 했거나 Push 했다면 삭제하거나 main 에 합쳐 해결하려 하지 않습니다. 현재 브랜치와 커밋 제목을 프로그래머에게 알려 안전하게 옮겨 달라고 요청합니다.

Settings.json 만 계속 남아요

체크를 해제했으므로 커밋 뒤에도 남는 것이 정상입니다. 다른 변경이 섞여 있지 않은지 먼저 확인합니다. 매번 커밋에 넣지 말고 자동 변경 파일 처리 방법을 프로그래머에게 확인합니다.

브랜치나 저장소가 목록에 없어요

Current repository 가 맞는지 먼저 확인합니다. Fetch origin 뒤 Current branch 검색 칸에 정확한 브랜치 이름을 입력합니다. 그래도 없다면 브랜치 이름, 접근 권한, 상대가 Publish branch 또는 Push 를 마쳤는지 확인합니다.

마지막 한 장: 작업 체크리스트

이 표를 작업할 때 옆에 두고 확인하세요. **현재 브랜치 확인 → Unity 저장 → Commit → Push** 가 핵심입니다.

시점	확인할 것	완료
시작	Current repository 가 CK_Semester_project 인가요?	☐
시작	Unity 를 닫고 Fetch origin 을 눌렀나요?	☐
시작	Pull origin 이 나왔다면 받기를 마쳤나요?	☐
브랜치	이번 작업의 기준 브랜치를 확인했나요?	☐

시점	확인할 것	완료
브랜치	Current branch 가 내 작업 브랜치인가요?	☐
브랜치	같은 브랜치·씬·프리팹의 담당자가 겹치지 않나요?	☐
작업 후	문서 2 대로 Unity 에서 저장했나요?	☐
작업 후	GitHub Desktop 으로 돌아오기 전에 Unity 를 닫았나요?	☐
Commit 전	Changes 에서 내가 바꾼 파일과 필요한 .meta 를 확인했나요?	☐
Commit 전	ProBuilder 의 Settings.json 체크를 해제했나요?	☐
Commit 전	큰 파일과 모르는 자동 변경 파일을 확인했나요?	☐
Commit	Summary 를 종류 : 변경 내용으로 적었나요?	☐
Commit	버튼의 대상 브랜치를 확인하고 Commit 했나요?	☐
전송	Publish branch 또는 Push origin 을 오류 없이 마쳤나요?	☐
마무리	History 에 내 커밋이 있고 전송 버튼 상태도 확인했나요?	☐
인계	브랜치·커밋·작업 구역을 프로그래머에게 전달했나요?	☐

> **팁** **Unity 에서 저장한 뒤, Commit 다음 Push 까지 해야 끝입니다.** 오류나 충돌이 남아 있다면 끝난 것으로 보지 말고 프로그래머에게 알려 주세요.